LiquidHaskell: Experience with Refinement Types in the Real World

Niki Vazou, Eric L. Seidel, Ranjit Jhala

Szabó Márk

Haskell Background

Haskell's Type System

Haskell is a **statically typed** language.

- Every expression has a type.
- Types are checked before the program runs.
- Types are often inferred automatically.

Basic Types

Some common built-in types:

```
Int        -- integer
Float      -- floating point number
Double     -- floating point number
Num        -- [Int, Float, Double, ...]
Bool       -- True or False
Char       -- single character
String     -- text
```

Type Annotations

The operator `::` means “has type”.

```
age :: Int
age = 23

name :: String
name = "Mark"

active :: Bool
active = True
```

Function Types

The arrow (→) separates input and output types.

```
add :: Int -> Int -> Int
add x y = x + y
```

- First Int: first argument
- Second Int: second argument
- Last Int: result

Every function takes one argument: `Int → Int → Int` means `Int → (Int → Int)`, so `add 1` is a function `Int → Int`.

Roughly equivalent (uncurried) in a C-like language:

```
int add(int x, int y) { return x + y; }
```

Type Inference

Haskell can infer many types automatically.

```
1 age = 23
2 double x = x * 2
```

In GHCi we can inspect inferred types:

```
1 :t True
2 True :: Bool
3
4 :t "hello"
5 "hello" :: String
6
7 :t double
8 double :: Num a => a -> a
```

Limits of The Type System

We can’t define constraints for the types of a function like:

- Conditions about the value: `x > 0`
- Relationships between values: `a > b`

To check for positive we need to check inside the function:

```haskell
sqrtRoot :: Double -> Double
sqrtRoot x
  | x > 0 = sqrt x
  | otherwise = error "x must be greater than 0"
```

Refinement Types and LiquidHaskell

LiquidHaskell

Many of these properties and relationships can be verified using a combination of **Refinement Types** and **external SMT solvers**.

This paper introduces and experiments with **LiquidHaskell** a refinement typechecker used to verify more than **10.000** lines of Haskell code across popular libraries.

Including: *bytestring*, *text*, *vector-algorithms* and *xmonad*.

Refinement Types

Refinement types:

- contain a **predicate** which is assumed to be true for any element of the refined type.
- can express **preconditions** when used as function arguments or **postconditions** when used as return types.

Examples:

positive natural numbers

$x : \{n \in \mathbb{N} \mid n > 0\}$

even natural numbers

$x : \{n \in \mathbb{N} \mid n \bmod 2 = 0\}$

result is a positive number

$f : \mathbb{N} \to \{y \in \mathbb{N} \mid y > 0\}$

result > input

$f(x : \mathbb{N}) : \mathbb{N} \to \{y \in \mathbb{N} \mid y > x\}$

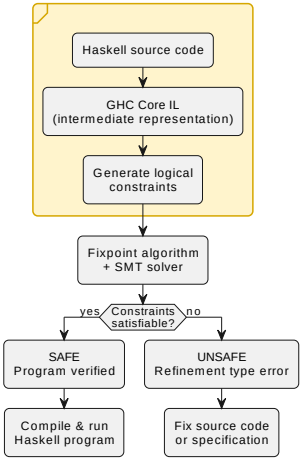
LiquidHaskell Architecture

Core

- Core is a simpler intermediate representation (GHC translates Haskell source to it)
- Source locations are preserved for error reporting

Constraints

- Refinement types are translated into logical constraints
- These constraints describe the properties that must hold



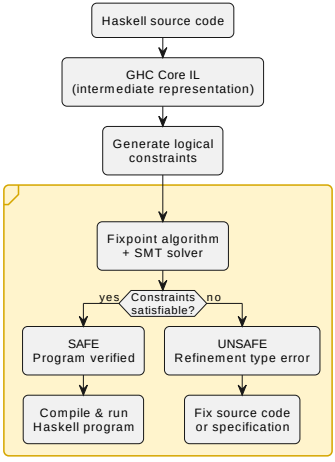
LiquidHaskell Architecture

Solution

- A fixpoint algorithm processes the constraints
- Any *SMT-LIB2* compatible solver checks whether the constraints can be satisfied (eg.: *Z3*, *CVC4*, …)

Result

- **SAFE** or **UNSAFE**: based on the verification of the refinements
- Errors are mapped back to their source-code locations



LiquidHaskell

LiquidHaskell is an **optional** type checker for Haskell. Refinement types are written inside **comments**, so they are optional and normal Haskell compilers can ignore them:

```
1 {-@ ... @-}
```

Execution

LiquidHaskell performs static verification before the Haskell program is executed. It does not become part of the executable or perform any checks during runtime.

LiquidHaskell Basics

```
1  {-@ type Natural = {v:Int | v >= 0} @-}
2  {-@ type Pos = {v:Int | v > 0} @-}
3
4  {-@ increment :: Natural -> Pos @-}
5  increment :: Int -> Int
6  increment x = x + 1
7
8  good1 = increment 0
9  good2 = increment 4
10
11 -- verification will fail here stating: Type Mismatch
12 bad = increment (-1)
```

LiquidHaskell Basics

```
1  {-@ type Natural = {v:Int | v >= 0} @-}
2  {-@ type Pos = {v:Int | v > 0} @-}
3
4  {-@ divide :: n:Natural -> d:Pos -> {v:Natural | v <= n} @-}
5  divide :: Int -> Int -> Int
6  divide x y = x `div` y
7
8  good = divide 5 2
9
10 -- With LH -> Type Mismatch before executing the code
11 -- Without LH -> runtime error: divide by zero
12 bad = divide 5 0
```

LiquidHaskell Parameterized Types

Type parameters are written as lowercase and **value parameters** are written as uppercase.

- **Type parameter:** *Int*, *Bool*, *[Int]*
- **Value parameter:** *0*, *10*, *100*

```
1 {-@ type NonEmpty a = {v:[a] | len v > 0} @-}
2
3 {-@ numbers :: NonEmpty Int @-}
4 numbers :: [Int]
5 numbers = []
6
7 {-@ letters :: NonEmpty Char @-}
8 letters :: [Char]
9 letters = ['a', 'b', 'c']
```

LiquidHaskell Parameterized Types

```
1  {-@ type Rng Low High = {v:Int | v >= Low && v <= High} @-}
2
3  {-@ grade :: Rng 0 100 -> Int @-}
4  grade :: Int -> Int
5  grade score
6      | score >= 90 = 5
7      | score >= 75 = 4
8      | score >= 60 = 3
9      | score >= 50 = 2
10     | otherwise   = 1
11
12 good = grade 85
13 bad = grade 120
```

LiquidHaskell Parameterized Types

```
1  -- a: element type, N list length
2  {-@ type ListN a N = {v:[a] | len v == N} @-}
3
4  {-@ numbers :: ListN Int 3 @-}
5  numbers :: [Int]
6  numbers = [1, 2, 3]
7
8  {-@ letters :: ListN Char 2 @-}
9  letters :: [Char]
10 letters = ['a', 'b']
11
12 {-@ invalid :: ListN Bool 3 @-}
13 invalid :: [Bool]
14 invalid = [True, False]
```

LiquidHaskell Type Aliases

Type Aliases: used for reusability and readability.

```
1  -- with type alias
2  {-@ predicate Between Low High V = V >= Low && V <= High @-}
3  {-@ type Rng Low High = {v:Int | Between Low High v} @-}
4
5  -- we can use the predicate inline as well
6  {-@ grade :: Rng 0 100 -> {v:Int | Between 1 2 v} @-}
7  grade :: Int -> Int
8  grade score
9      | score >= 50 = 2
10     | otherwise   = 1
11
12 good = grade 85
13 bad = grade 120
```

Contracts and Verification

LiquidHaskell Function Contracts

- **Precondition:** Constraint on the function's inputs.
- **Postcondition:** Constraint on the function's output.
- **Dependent types:** Output constraints can refer to input values.

```
1  {-@ type Rng Low High = {v:Int | v >= Low && v <= High} @-}
2
3  {-@ range :: lo:Int
4            -> hi:{Int | hi >= lo}
5            -> [Rng lo hi] @-}
6  range :: Int -> Int -> [Int]
7  range lo hi = [lo..hi]
8
9  -- [1..5] -> [1, 2, 3, 4, 5]
10 -- [5..1] -> []
```

LiquidHaskell Verification: Contracts

```
1  {-@ type Rng Low High = {v:Int | v >= Low && v <= High} @-}
2
3  {-@ range :: lo:Int
4            -> hi:{Int | lo <= hi}
5            -> [Rng lo hi] @-}
6  range :: Int -> Int -> [Int]
7  range lo hi = [lo..hi]
8
9  good = range 1 5
10 bad = range 5 1
```

- **SAFE:** range 1 5 satisfies the precondition $1 \leq 5$.
- **UNSAFE:** range 5 1 violates the precondition $5 \not\leq 1$.

LiquidHaskell Verification: Limitations

- Haskell's `[lo..hi]` is alias for `enumFromTo lo hi`.
- LiquidHaskell relies on refinement specifications to reason about library functions.
- Without a sufficiently precise specification for `enumFromTo`, it cannot prove that every element satisfies `Rng lo hi`.

```
1 -- Cannot verify the postcondition
2 range lo hi = [lo..hi]
```

Important

UNSAFE does not necessarily mean the code is wrong. It can mean LiquidHaskell cannot prove its correctness.

LiquidHaskell Verification: Recursive Range

```
1 {-@ range :: lo:Int
2           -> hi:{Int | lo <= hi}
3           -> [Rng lo hi] @-}
4 range :: Int -> Int -> [Int]
5 range lo hi
6   | lo <= hi = lo : range (lo + 1) hi
7   | otherwise = []
```

- At `lo == hi`, the recursive call is `range (hi + 1) hi`.
- Its precondition requires $hi + 1 \leq hi$, which is false.

LiquidHaskell Verification: Fixing the Boundary

```
{-@ range :: lo:Int
          -> hi:{Int | lo <= hi}
          -> [Rng lo hi] @-}
range :: Int -> Int -> [Int]
range lo hi
  | lo < hi   = lo : range (lo + 1) hi
  | otherwise = [lo]
```

- When `lo < hi`, integer arithmetic gives $lo + 1 \leq hi$.
- Otherwise, the precondition implies `lo == hi`; return the endpoint.
- Every returned element satisfies $lo \leq v \leq hi$.
- By default, LiquidHaskell tries to prove termination using a **decreasing argument**, but here `lo` increases instead.

LiquidHaskell Termination

Haskell only evaluates expressions when their results are needed (*non-strict evaluation*).

- Some functions may run forever without producing a result.
- **LiquidHaskell:** Must ensure that functions terminate to safely verify certain properties.
- By default, LiquidHaskell proves termination of each recursive function.

Termination metric

A termination metric is a value that:

- Gets smaller with **every** recursive call.
- Cannot decrease forever (e.g., a non-negative integer).

LiquidHaskell Termination: Factorial

A simple termination metric is a **non-negative integer**.

```
1 {-@ fac :: n:Nat -> Nat / [n] @-}
2 fac :: Int -> Int
3 fac 0 = 1
4 fac n = n * fac (n - 1)
```

- `Nat`: integers greater than or equal to zero.
- `/ [n]`: use the argument `n` as the termination metric.
- The base case returns immediately.
- The recursive case decreases `n` by 1 each call.

LiquidHaskell Termination: Proof Obligations

Refinement checking establishes the facts needed for `fac`:

- The input contract gives $n \geq 0$.
- In the recursive branch, $n \neq 0$, so $n > 0$.
- The next argument remains non-negative: $n - 1 \geq 0$.
- The metric strictly decreases: $n - 1 < n$.

Why this proves termination

A strictly decreasing sequence of non-negative integers cannot continue forever. At zero, the base case stops recursion.

LiquidHaskell Verification: Proving Termination

```
1 {-@ range :: lo:Int
2           -> hi:{Int | lo <= hi}
3           -> [Rng lo hi] / [hi - lo] @-}
4 range :: Int -> Int -> [Int]
5 range lo hi
6   | lo < hi   = lo : range (lo + 1) hi
7   | otherwise = [lo]
```

- `/ [hi - lo]` supplies an explicit **termination metric**.
- The precondition ensures $hi - lo \geq 0$.
- Each recursive call strictly decreases the metric:

$$hi - (lo + 1) = (hi - lo) - 1.$$

- At `lo == hi`, the metric is zero and recursion stops.

Refining Data and Interfaces

LiquidHaskell Measures

So far, the specifications have been limited to comparisons and arithmetic operations on **primitive** values. **Measures** describe **inductive properties** of algebraic data types.

```
1 {-@ measure myLen @-}
2 myLen :: [a] -> Int
3 myLen []     = 0
4 myLen (x:xs) = 1 + myLen xs
```

- One equation for each data constructor: `[]` and `(:)`.
- Each right-hand side must belong to the restricted refinement logic.
- `measure myLen` makes this Haskell function available in refinements; it must obey the measure restrictions.

LiquidHaskell Measures: Minimum Length

```
1 {-@ type MinLength a N = {v:[a] | myLen v >= N} @-}
2
3 {-@ good :: MinLength Int 3 @-}
4 good :: [Int]
5 good = [1, 2, 3, 4]
6
7 {-@ bad :: MinLength Int 3 @-}
8 bad :: [Int]
9 bad = [1, 2]
```

- `a` is the element type; `N` is the minimum length.
- **SAFE:** `good` has length 4, and $4 \geq 3$.
- **UNSAFE:** `bad` has length 2, and $2 < 3$.

LiquidHaskell Measures: Constructor Refinements

Each list constructor gives LiquidHaskell a **length fact**:

```
[]  :: {v:[a] | myLen v == 0}

(:) :: x:a -> xs:[a]
    -> {v:[a] | myLen v == 1 + myLen xs}
```

- `[]`: the length is zero.
- `x:xs`: the length is one more than `xs`.
- These facts are available when building or pattern matching on lists.

LiquidHaskell Measures: Using a Constructor Fact

Adding an element increases the minimum length.

```
1 {-@ prepend :: a -> MinLength a 2 -> MinLength a 3 @-}
2 prepend :: a -> [a] -> [a]
3 prepend x xs = x : xs
4
5 good = prepend 1 [2, 3]
6 bad = prepend 1 [2]
```

- Input contract: $myLen\ xs \geq 2$.
- Constructor fact: $myLen\ (x : xs) = 1 + myLen\ xs \geq 3$.

LiquidHaskell Measures: Function Contracts

Measures relate the **structure of inputs and outputs**.

```
1  -- Appending adds the lengths
2  {-@ append :: xs:[a] -> ys:[a]
3             -> {v:[a] | myLen v == myLen xs + myLen ys} @-}
4
5  -- Mapping preserves the length
6  {-@ map :: (a -> b) -> xs:[a]
7          -> {v:[b] | myLen v == myLen xs} @-}
8
9  -- Filtering cannot increase the length
10 {-@ filter :: (a -> Bool) -> xs:[a]
11            -> {v:[a] | myLen v <= myLen xs} @-}
```

LiquidHaskell Refined Data Types

Every value of a data type may need to satisfy an **invariant**.

```
1 data CSV a = CSV { cols :: [String], rows :: [[a]] }
```

- `cols`: the column names in the header.
- `rows`: the table's rows, containing values of type `a`.
- Haskell allows rows of different lengths.

CSV invariant

Every row must have exactly as many entries as the header has column names.

LiquidHaskell Refined Data Types: Definition

A field's refinement can refer to an **earlier field**.

```
1 {-@ type ListL a X = {v:[a] | len v == len X} @-}
2
3 {-@ data CSV a = CSV { cols :: [String]
4                     , rows :: [ListL a cols] } @-}
```

- `ListL a X`: a list with the same length as `X`.
- `rows`: a list of rows, each matching the length of `cols`.
- `len` is LiquidHaskell's built-in list length measure.

LiquidHaskell Refined Data Types: Verification

```
1 good = CSV ["Month", "Days"]
2              [ ["Jan", "31"]
3              , ["Feb", "28"]
4              , ["Mar", "31"] ]
5
6 bad =  CSV ["Month", "Days"]
7             [ ["Jan", "31"]
8             , ["Feb", "28"]
9             , ["Mar"] ]
```

LiquidHaskell Refined Data Types: Global Invariants

Like measures, refined data definitions strengthen **constructor types**.

```
1 -- Refined constructor type
2 CSV :: cols:[String] -> rows:[ListL a cols] -> CSV a
```

- Construction checks every row against the header length.
- Pattern matching makes this relationship available to the checker.
- No extra annotation is needed on individual CSV values.

Global guarantee: LiquidHaskell rejects CSV-typed expressions that violate this invariant.

LiquidHaskell Refined Type Classes

A **type class** defines a shared interface for different types.

```
1 class Reward a where
2   reward :: a -> Int -> Int
```

- `a`: the type that determines the reward policy.
- `reward`: takes a policy value and the current points.
- Each instance provides its own implementation.

Shared rule: points must be non-negative and cannot decrease.

LiquidHaskell Refined Type Classes: Contract

Refinements specify a **shared contract** for the class method.

```
{-@ class Reward a where
      reward :: a -> points:{Int | points >= 0}
             -> {v:Int | v >= points}
@-}
```

- **Precondition:** input points must be non-negative.
- **Postcondition:** the result cannot be smaller than the input.
- Every instance must satisfy the **same contract**.

LiquidHaskell Refined Type Classes: Bool Instance

A Boolean flag decides whether to add a bonus.

```
1 instance Reward Bool where
2   reward bonus points
3     | bonus     = points + 10
4     | otherwise = points
```

- `True`: add 10 points.
- `False`: keep the original points.
- Both branches satisfy $v \geq \mathit{points}$.

LiquidHaskell checks the instance against the class contract.

LiquidHaskell Refined Type Classes: Verification

```
1 good :: Int
2 good = reward True 20
3
4 bad :: Int
5 bad = reward True (-1)
```

- **SAFE:** `good` passes non-negative points and returns 30.
- **UNSAFE:** `bad` violates the precondition $points \geq 0$.

LiquidHaskell Refined Type Classes: Invalid Instance

What if we replace the bonus with a deduction?

```
1 -- alternative implementation:
2 instance Reward Bool where
3   reward bonus points
4     | bonus     = points - 10
5     | otherwise = points
```

- The first branch violates $v \geq points$.
- Callers must meet the precondition.
- Instances must prove the postcondition.

Totality and Advanced Termination

LiquidHaskell Totality: Missing Patterns

A function can pass Haskell's type checker but still fail on some inputs.

```
1 head :: [a] -> a
2 head (x:_) = x
3
4 good = head [1, 2, 3]
5 bad = head []
```

- The definition only handles non-empty lists.
- An empty list causes a **pattern-match exception**.
- Totality checking must prove that this failure cannot happen.

LiquidHaskell Totality: Core Translation

GHC makes the missing case explicit in Core:

```
1 head d = case d of
2   x:xs -> x
3   []   -> patError "head"
```

- Both list constructors now have a branch.
- `patError` throws an exception describing the failed match.
- To rule out this exception, prove that the error branch is **unreachable**.

LiquidHaskell Totality: Making Head Safe

Restrict the input to **non-empty lists**.

```
1 {-@ type NonEmpty a = {v:[a] | len v > 0} @-}
2 {-@ head :: NonEmpty a -> a @-}
3 head :: [a] -> a
4 head (x:_) = x
```

- The input contract gives $len\ v > 0$.
- The empty branch gives $len\ v = 0$: a contradiction.
- The error call is unreachable.

LiquidHaskell Termination: Processing Batches

Consider a simplified **batch worker** that counts completed jobs.

- `b`: batches remaining, including the current batch.
- `j`: jobs remaining in the current batch.
- Completing a job decreases `j`, while `b` stays unchanged.
- Moving to the next batch decreases `b` and resets `j` to 3.

Two batches of three jobs

$(2,3) \to (2,2) \to (2,1) \to (2,0) \to (1,3) \to \cdots \to (1,0)$.

Neither argument strictly decreases at every call. We supply a **custom termination metric**: the pair `[b, j]`.

LiquidHaskell Termination: Lexicographic Ordering

Compare the **batch count first**, then the job count.

```
1 {-@ jobs :: b:Nat -> j:Nat -> Nat / [b, j] @-}
2 jobs :: Int -> Int -> Int
3 jobs b j
4   | b == 0    = 0
5   | j > 0     = 1 + jobs b (j - 1)
6   | b == 1    = 0
7   | otherwise = jobs (b - 1) 3
8 -- jobs 2 3 == 6
```

LiquidHaskell Termination: Why the Pair Decreases

`/ [b, j]` uses **lexicographic ordering**: compare `b` first, and compare `j` only when `b` is unchanged.

- `jobs b (j - 1)`: the first component stays equal, the second decreases.
- `jobs (b - 1) 3`: the first component decreases.
- Both components remain non-negative.
- At `b == 0`, or when the last batch is empty (`b == 1` and `j == 0`), recursion stops.

Memory Safety

LiquidHaskell Memory Safety

Haskell's high-level libraries can use **unsafe memory operations** internally for speed.

- **ByteString:** stores bytes and uses pointer arithmetic.
- Refinements track buffer sizes and valid positions.

Main idea

Before reading or writing memory, prove that there is enough space.

LiquidHaskell ByteString: Staying Inside the Buffer

A ByteString is a **slice**: a pointer, a starting offset, and a length.

```
measure fplen :: ForeignPtr a -> Int
data ByteString = PS
 { pay :: ForeignPtr Word8
 , off :: {v:Nat | v       <= (fplen pay)}
 , len :: {v:Nat | off + v <= (fplen pay)} }
```

- `fplen pay`: the buffer size in bytes.
- `off + len` cannot exceed that size.

Paper notation: refinements are shown directly, without comment wrappers.

LiquidHaskell ByteString: Pointer Contracts

A pointer with room for **at least N bytes**:

```
1 type PtrN a N = {v:Ptr a | N <= (plen v)}
```

- `plen v`: the available buffer size.
- Copying N bytes requires that much space in both buffers.
- LiquidHaskell checks these requirements at the call site.

Trusted boundary

Contracts for external C functions are assumed, their C code is not verified.

These contracts ensure that copying stays inside both buffers.

Functional Correctness Invariants

LiquidHaskell Functional Correctness

Beyond avoiding errors, refinements can express **application-specific correctness rules**.

The paper's example: **red-black trees**, a balanced binary search tree structure.

```
1 data Col    = R | B
2 data Tree a = Leaf
3             | Node Col a (Tree a) (Tree a)
```

- `Col`: the color; `a`: the key; followed by the left and right subtrees.

LiquidHaskell Red-Black Trees: Three Invariants

A tree is only a valid Red-Black tree if:

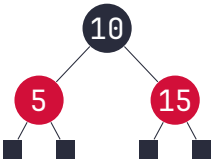
- **Order:** all keys in the left subtree are smaller than the node's keyand all keys in the right subtree are larger.
- **Color:** a red node has only black children. Empty leaves count as black.
- **Height:** from any node, every path to a leaf contains the same number of black nodes.

Main idea

Describe these properties separately, then combine them without changing the underlying tree structure.

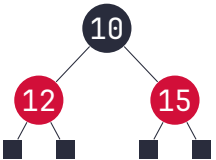
LiquidHaskell Red-Black Trees: Valid and Invalid

Valid



$5 < 10 < 15$.
Color and height are valid too.

Invalid



Order fails: $12 > 10$.

Black squares represent empty leaves.

LiquidHaskell Red-Black Trees: Checking Color

```
1 measure isB :: Tree a -> Prop
2 isB (Node c x l r) = c == B
3 isB (Leaf) = true
4
5 measure isRB :: Tree a -> Prop
6 isRB (Leaf) = true
7 isRB (Node c x l r) = isRB l && isRB r &&
8                       c == R => (isB l && isB r)
```

- `isB`: the root is black; empty leaves count as black.
- `isRB`: red nodes have black children, recursively.

LiquidHaskell Red-Black Trees: Checking Height

```
1 measure bh :: Tree a -> Int
2 bh (Leaf) = 0
3 bh (Node c x l r) = bh l + if c == R then 0 else 1
4
5 measure isBal :: Tree a -> Prop
6 isBal (Leaf) = true
7 isBal (Node c x l r) = bh l == bh r && isBal l && isBal r
```

- `bh`: count black nodes along the leftmost path.
- `isBal`: require equal black heights, recursively.

LiquidHaskell Red-Black Trees: Checking Order

```
1  data Tree a <l :: a -> a -> Prop,
2              r :: a -> a -> Prop>
3    = Leaf
4    | Node { c   :: Col
5           , key :: a
6           , lt  :: Tree <l, r> a <l key>
7           , rt  :: Tree <l, r> a <r key> }
8  type OTree a
9    = Tree <{\k v -> v < k},   -- l: left rule
10          {\k v -> v > k}> a -- r: right rule
```

- `l` and `r`: rules relating keys to subtree values.
- `OTree`: left keys $v < k$; right keys $v > k$.

LiquidHaskell Red-Black Trees: Combining the Rules

Combining ordering, color, and black-height balance:

```
1 type RBT a = {v:OTree a | isRB v && isBal v}
```

- `OTree`: keys follow binary-search ordering.
- `isRB`: red nodes have black children.
- `isBal`: all paths have equal black heights.

With these invariants and suitable function signatures, LiquidHaskell verifies the library's **insertion, deletion, and rebalancing** procedures.

LiquidHaskell Stack Sets in XMonad

xmonad is a dynamically tiling X11 window manager, written and configured in Haskell.

- The managed windows are organized into a **hierarchy of types**.
- Every level stores windows, together with layout and screen data.

Key invariant

Each window appears **at most once**: it is never duplicated across stacks or workspaces.

LiquidHaskell XMonad: The Stack Zipper

At the lowest level, windows are stored in a **zipper**:

```
1 data Stack a = Stack { focus :: a
2                      , up    :: [a]
3                      , down  :: [a] }
```



- `focus`: the current window.
- `up` and `down`: the windows before and after it.

LiquidHaskell XMonad: Type Hierarchy

```haskell
data Workspace i l a = Workspace
  { tag :: i, layout :: l, stack :: Maybe (Stack a) }

data Screen i l a sid sd = Screen
  { workspace :: Workspace i l a
  , screen :: sid
  , screenDetail :: sd }

data StackSet i l a sid sd = StackSet
  { cur :: Screen i l a sid sd       -- focused screen
  , vis :: [Screen i l a sid sd]     -- other visible screens
  , hid :: [Workspace i l a]         -- hidden workspaces
  , flt :: M.Map a RationalRect }    -- floating windows
```

LiquidHaskell XMonad: Unique Lists

A measure collects the elements of a list into a **set**:

```
1 measure elts :: [a] -> Set a
2 elts ([])   = emp
3 elts (x:xs) = cup (sng x) (elts xs)
4
5 measure isUniq :: [a] -> Prop
6 isUniq ([])   = true
7 isUniq (x:xs) = notIn x xs && isUniq xs
8
9 predicate notIn X S = not (mem X (elts S))
```

- `emp`, `sng`, `cup`: empty set, singleton, union ($\emptyset$, $\{x\}$, $\cup$).
- `mem`: set membership ($\in$).
- `isUniq`: no element occurs in the rest of the list.

LiquidHaskell XMonad: Unique Stacks

```
1 data Stack a = Stack
2   { focus :: a
3   , up    :: {v:[a] | Uniq1 v focus}
4   , down  :: {v:[a] | Uniq2 v focus up} }
5
6 predicate Uniq1 V X   = isUniq V && notIn X V
7 predicate Uniq2 V X Y = Uniq1 V X && disjoint Y V
8 predicate disjoint X Y = cap (elts X) (elts Y) = emp
```

- `cap`: set intersection (∩); `disjoint` means no shared elements.
- `up`: a unique list that does not contain `focus`.
- `down`: the same, and also disjoint from `up`.

LiquidHaskell XMonad: Unique Stack Sets

`elts` and `isUniq` are lifted to `Stack`, `Workspace` and `Screen`.

```
1 type UniqStackSet i l a sid sd
2   = {v:StackSet i l a sid sd | NoDups v}
3
4 predicate NoDups V = disjoint3 (hid V) (cur V) (vis V)
5                   && isUniq (vis V) && isUniq (hid V)
6
7 predicate disjoint3 X Y Z
8   = disjoint X Y && disjoint Y Z && disjoint X Z
```

- Record selectors (`cur`, `vis`, `hid`) automatically become measures.
- No window is shared between the current, visible and hidden parts.

LiquidHaskell XMonad: Strengthening Library Functions

xmonad often concatenates `up s ++ down s`. Proving the result unique needs a stronger type for `(++)`:

```
1 -- assumed: two unique, disjoint lists give a unique list
2 (++) :: xs:{[a] | isUniq xs}
3       -> ys:{[a] | isUniq ys && disjoint xs ys}
4       -> {v:[a] | isUniq v}
5
6 "StackSet" ++ "error"   -- violates the precondition
```

- Not every call concatenates unique, disjoint lists.
- LiquidHaskell has no **intersection types**, so an unrefined `(++.)` is used for those cases.

LiquidHaskell XMonad: Restricting the Domain

Given a default value `x`, a function `f` and a stack set `s`, `modify` applies `f` to the `Maybe (Stack a)` values inside `s`.

```
1 modify :: Maybe (Stack a)                  -- x: default
2        -> (Stack a -> Maybe (Stack a))     -- f: update
3        -> StackSet i l a s sd              -- s: input
4        -> StackSet i l a s sd
```

- It should not introduce a **duplicate window**.

LiquidHaskell XMonad: Restricting the Domain

The refined type only accepts arguments that preserve uniqueness:

```
1 modify :: x:{v:Maybe (Stack a) | isNothing v}
2        -> (y:Stack a -> Maybe {v:Stack a | SubElts v y})
3        -> UniqStackSet i l a s sd
4        -> UniqStackSet i l a s sd
```

- `f`: the new stack may only contain windows of `y`: $elts\ v \subseteq elts\ y$.
- `x`: its windows could already exist elsewhere in `s`, so the only safe default is `Nothing`.
- Result: `s` stays a `UniqStackSet`.

Precondition: the default value must be `Nothing`.

LiquidHaskell XMonad: Summary

Verifying the uniqueness invariant required changes to the code and its specifications:

- **Strengthening library functions:** a stronger type for `(++)`, plus an unrefined `(++.)` for other uses.
- **Restricting the domain:** `modify` only accepts `Nothing` as its default.

Takeaway

Real code sometimes needs **small rewrites** to verify.

Evaluation

LiquidHaskell Evaluation: Benchmarks

Library	Verified properties
GHC.List, Data.List	size properties of list operations
Data.Set.Splay	termination, well-ordered splay trees
Data.Map.Base	termination, binary-search ordered trees
HsColour	totality of all functions
XMonad	window uniqueness, some QuickCheck properties
ByteString	termination, memory safety, correctness
Text	pointer safety, correctness (found a bug)
Vector-Algorithms	safe vector accessing, indexing and slicing

LiquidHaskell Evaluation: Results

Module	LOC	Mod	Fun	Specs	Annot	Qualif	Time (s)
GHC.List	309	1	66	29 / 38	6 / 6	0 / 0	15
Data.List	504	1	97	15 / 26	6 / 6	3 / 3	11
Data.Map.Base	1396	1	180	125 / 173	13 / 13	0 / 0	174
Data.Set.Splay	149	1	35	27 / 37	5 / 5	0 / 0	27
HsColour	1047	16	234	19 / 40	5 / 5	1 / 1	196
XMonad.StackSet	256	1	106	74 / 213	3 / 3	4 / 4	27
ByteString	3505	8	569	307 / 465	55 / 55	47 / 124	294
Text	3128	17	493	305 / 717	52 / 54	49 / 97	499
Vector-Algorithms	1218	10	99	76 / 266	9 / 9	13 / 13	89
Total	11512	56	1879	977 / 1975	154 / 156	117 / 242	1336

Specs, Annot, Qualif: count / lines of type specifications, other annotations (invariants, termination hints) and qualifiers.

LiquidHaskell Evaluation: What the Numbers Show

- **11,512** lines of code in 56 modules, checked with **1,975** lines of specifications.
- Hints (annotations and qualifiers) add only **3.5%** of the code size.

Performance

The full suite takes about **22 minutes**, but checking a handful of functions in isolation is fast enough for **interactive use**.

LiquidHaskell Limitations

The case studies also showed limitations of LiquidHaskell.

- Ghost parameters
- Fixed-width integers and floating-point numbers
- Higher-order functions
- Functions as data
- Lazy binders
- Assumes
- Error messages

Workaround

In most cases, the code could be **altered slightly** to make verification possible.

LiquidHaskell Limitations: Ghost Parameters and Numbers

Ghost parameters

- Some values are not needed for the computation, only for the proof.
- They must still be passed as extra arguments.

Fixed-width integers and floating-point numbers

- Numbers are modelled with linear arithmetic and real numbers.
- Some precision is lost, e.g. bitwise operations can only be approximated.

References

- Niki Vazou, Eric L. Seidel, Ranjit Jhala. *LiquidHaskell: Experience with Refinement Types in the Real World*. Haskell Symposium, 2014.
- LiquidHaskell documentation: https://ucsd-progsys.github.io/liquidhaskell